

SEGURIDAD HARDWARE

Parte I

PRESENTADO POR:
Iluminada Baturone

ENTIDAD
Universidad de Sevilla

MÓDULO
Avanzado

CONTENIDOS

Necesidad de seguridad hardware

1. Soluciones con seguridad hardware
2. Generación y uso de claves criptográficas
3. Arranque seguro (secure boot)
4. Actualización segura de firmware
5. Almacenamiento sellado de información
6. Atestación (remota) de plataformas
7. Conclusiones

NECESIDAD DE SEGURIDAD HARDWARE

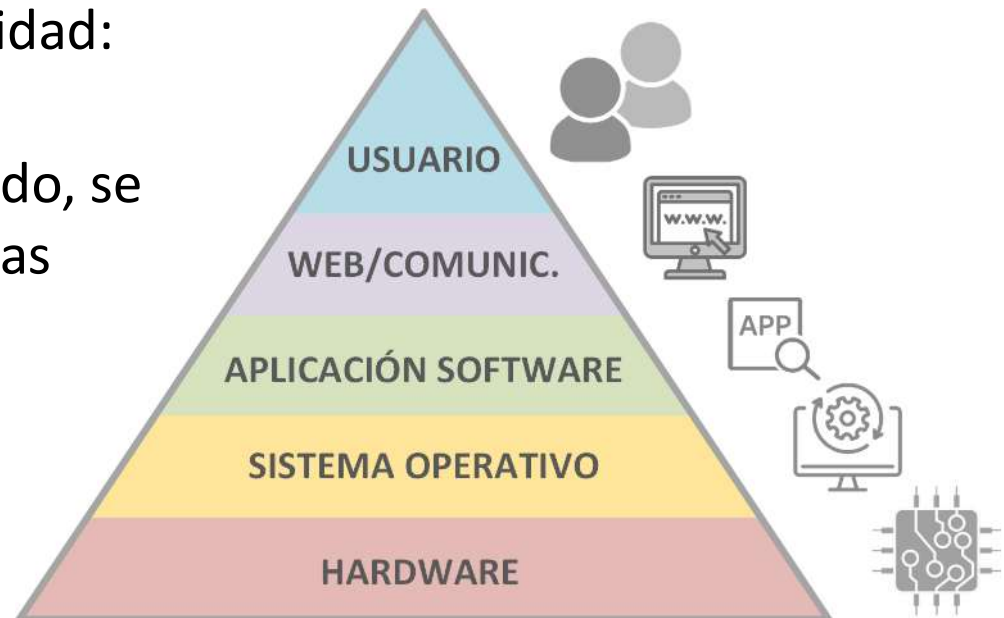
La seguridad de la información se basa en tres conceptos básicos: Confidencialidad, Integridad y Disponibilidad (CIA triad).

- **Confidencialidad**: garantizar el acceso solo a receptor autorizado
- **Integridad**: no alteración y autenticidad del mensaje y de su origen, y
- **Disponibilidad**: poder acceder a la información cuando sea requerida.

Niveles de abstracción para seguridad:

El hardware es la base. Si es atacado, se pueden inutilizar las defensas de las capas superiores.

La seguridad hardware ofrece el nivel más alto de confianza (high Level of Assurance, LoA)



CONTENIDOS

Necesidad de seguridad hardware

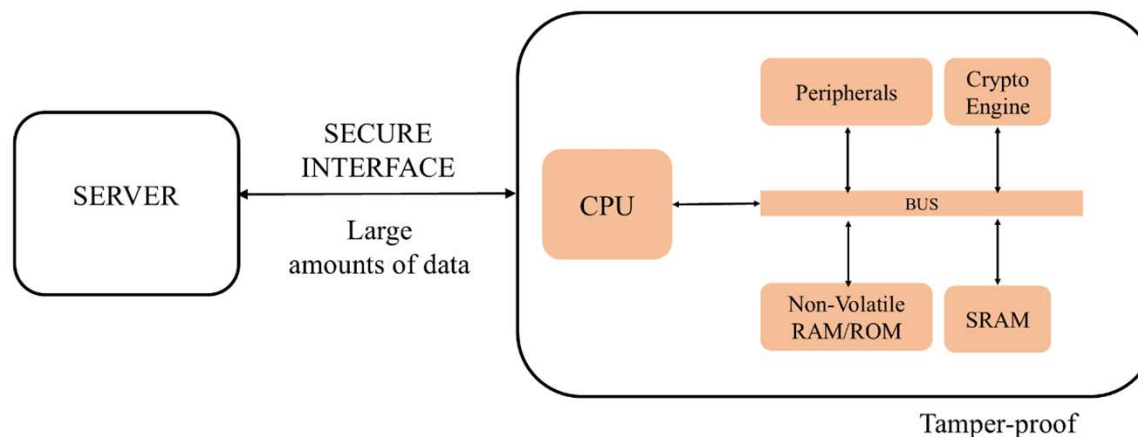
- 1. Soluciones con seguridad hardware**
2. Generación y uso de claves criptográficas
3. Arranque seguro (secure boot)
4. Actualización segura de firmware
5. Almacenamiento sellado de información
6. Atestación (remota) de plataformas
7. Conclusiones

NECESIDAD DE SEGURIDAD HARDWARE

Soluciones con seguridad hardware

Módulos de Seguridad Hardware (HSM)

- Plataforma especializada, con elevados recursos de memoria y cómputo (un cripto-procesador con módulos hardware dedicados, como TRNG y primitivas, como hash, cifrado/descifrado y firma).
- Las claves se generan dentro y nunca salen del dispositivo.



- En local, como Thales Luna, Ultimaco, Entrust nShield, ...
- En la nube (HSMaaS), como AWS CloudHSM, Azure Dedicated HSM, Google Cloud HSM,...

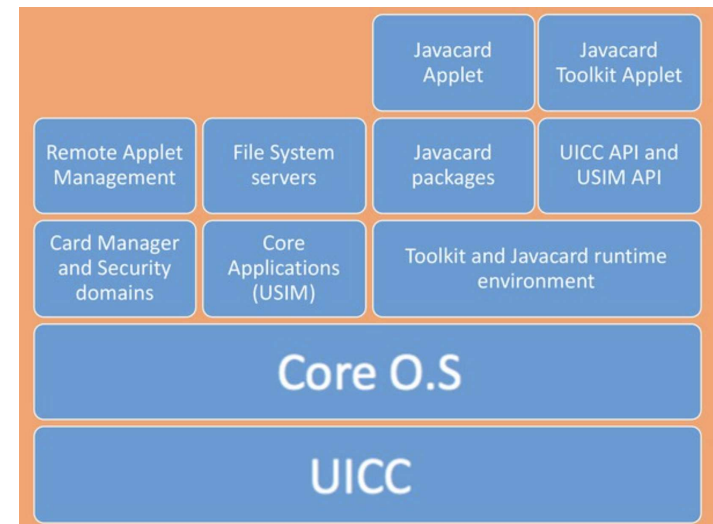
NECESIDAD DE SEGURIDAD HARDWARE

Soluciones con seguridad hardware

Elementos Seguros (SE)

Son chips especializados, con pocos recursos de memoria y cómputo:

- SE integrado (eSE), soldado en la placa del dispositivo (como los Samsung eSE en los smartphones Galaxy)
- SE integrado en el chip UICC (Universal Integrated Circuit Card) de la tarjeta SIM
- SE en tarjeta SD o USB externo (como YubiKey o BioPass)



- Pueden seguir la especificación TPM 2.0, estándar ISO/IEC 11889:2015 (Trusted Platform Module del Trusted Computing Group)

NECESIDAD DE SEGURIDAD HARDWARE

Soluciones con seguridad hardware

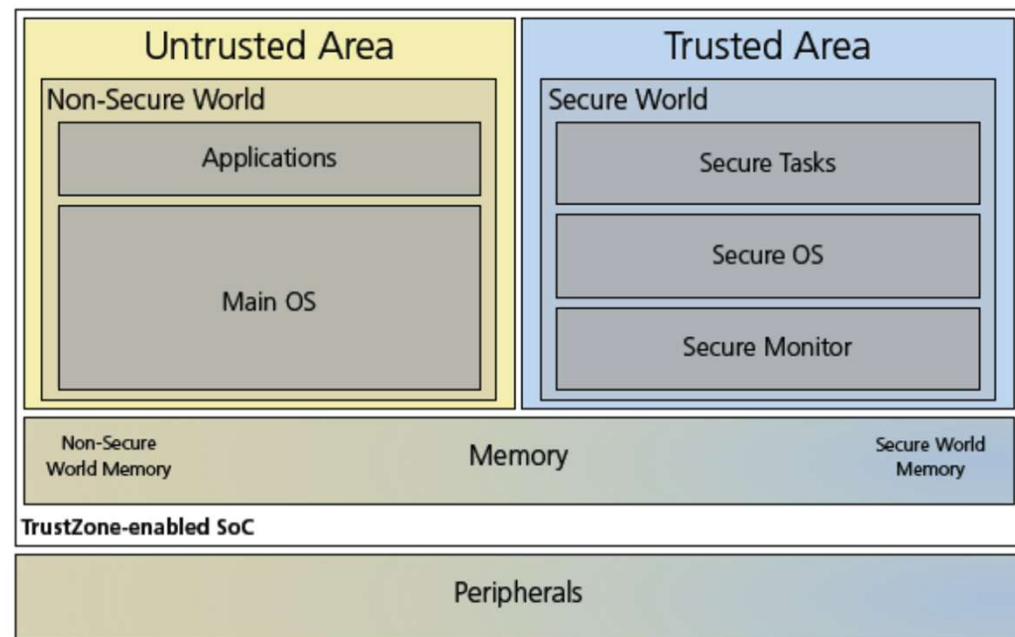
Entornos de Ejecución Segura (TEEs)

Extensiones arquitecturales de plataformas ya existentes:

- [ARM TrustZone](#) (integrado en los procesadores Qualcomm de muchos smartphones y en los Cortex-M de muchos dispositivos IoT)

Partición en dos mundos:
Seguro y Normal.

Introducción de un “Non-secure bit”.



NECESIDAD DE SEGURIDAD HARDWARE

Soluciones con seguridad hardware

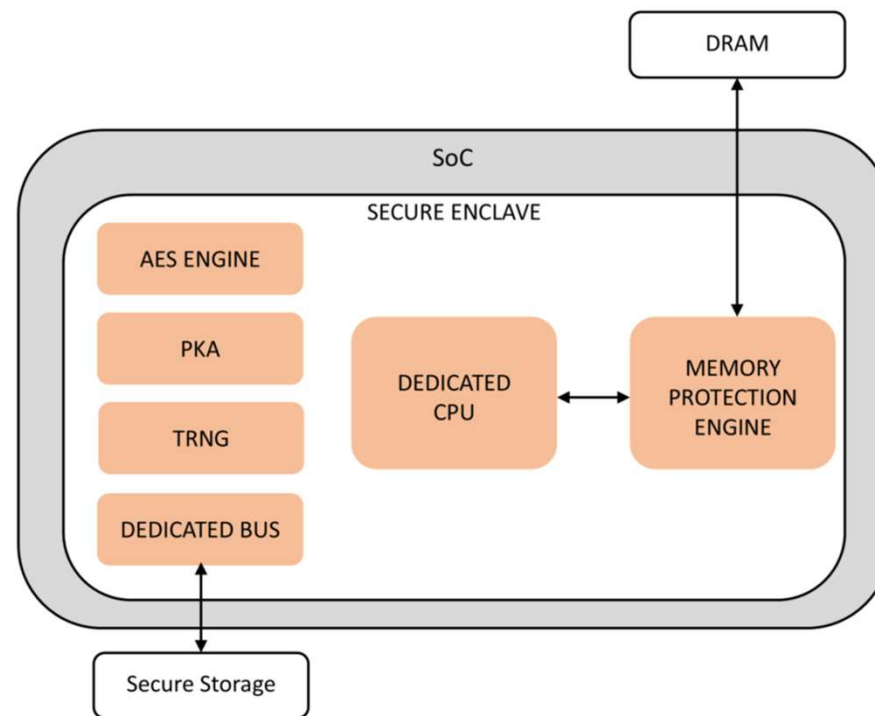
Entornos de Ejecución Segura (TEEs)

Extensiones arquitecturales de plataformas ya existentes:

- **Enclave Seguro de Apple** (integrado en los iPhone, iPad, Mac, Apple TV, Apple Watch, etc.)

Subsistema dentro del SoC.

Cifra una parte de la DRAM.



NECESIDAD DE SEGURIDAD HARDWARE

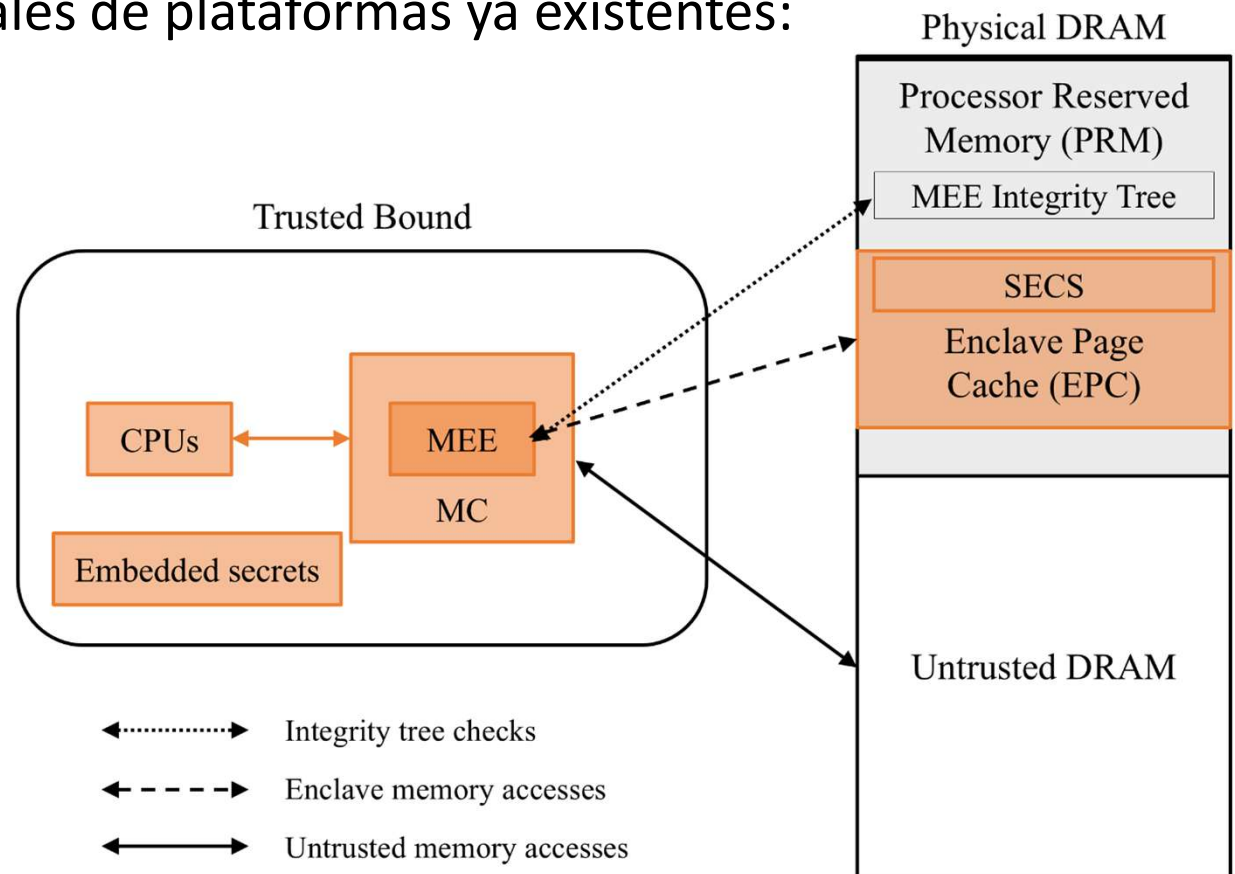
Soluciones con seguridad hardware

Entornos de Ejecución Segura (TEEs)

Extensiones arquitecturales de plataformas ya existentes:

- **Intel SGX** (integrado en ciertas CPUs de Intel como los Intel Xeon Scalable)

Permite crear enclaves seguros dentro de la memoria del sistema, aislados del sistema operativo y del hipervisor



NECESIDAD DE SEGURIDAD HARDWARE

Soluciones con seguridad hardware

En el ecosistema IoT las soluciones son más a medida de la aplicación

Restricciones en capacidad de cómputo y almacenamiento y consumo de potencia



CONTENIDOS

Necesidad de seguridad hardware

1. Soluciones con seguridad hardware
- 2. Generación y uso de claves criptográficas**
3. Arranque seguro (secure boot)
4. Actualización segura de firmware
5. Almacenamiento sellado de información
6. Atestación (remota) de plataformas
7. Conclusiones

NECESIDAD DE SEGURIDAD HARDWARE

Las claves criptográficas se necesitan para:

- **Almacenar** datos sensibles cifrados en una memoria no segura.

Seguridad “in rest”. Clave de sellado.

- **Transmitir** datos sensibles por canales inseguros

Seguridad “in transit”.

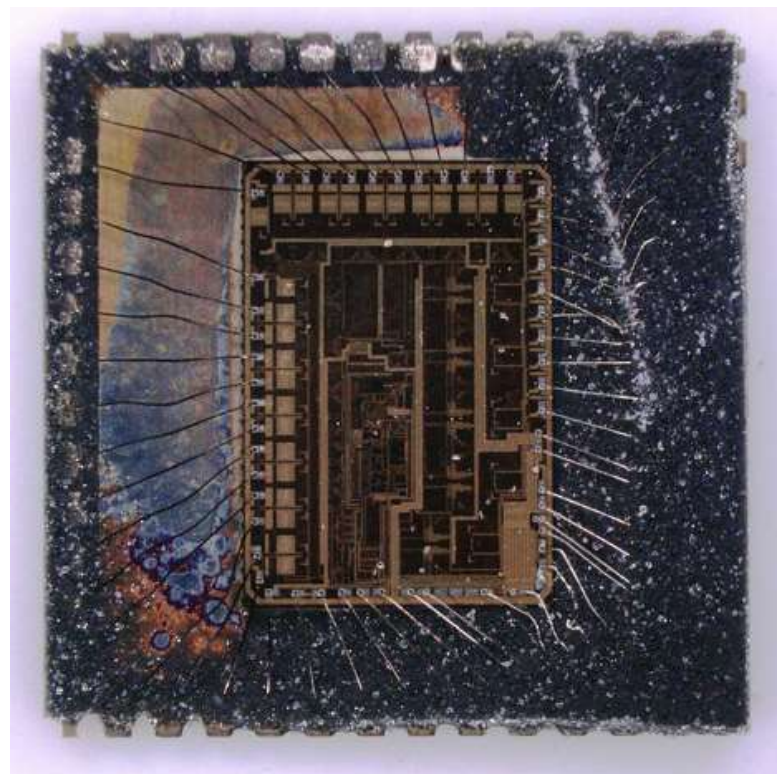
- **Autenticar** a una entidad frente a otras.

Clave de atestación.

NECESIDAD DE SEGURIDAD HARDWARE

Generación y uso de claves criptográficas

- Normalmente, las claves secretas se almacenan en una memoria situada en el interior del chip (ROM interna o flash).
- Inconveniente: ¡el chip se puede decapsular y aplicar microprobing a los buses de memoria interna!
- El almacenamiento es permanente dentro del chip, por lo que el tiempo para atacar es muy largo ...



Chip decapsulado

... ¿y si las claves secretas no se almacenan, sino que se recuperan cuando sean necesarias mediante un identificador del dispositivo?

NECESIDAD DE SEGURIDAD HARDWARE

Generación y uso de claves criptográficas

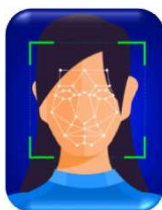
Se puede emplear **PUFs (Physical Unclonable Functions)** en el hardware.

Sus propiedades son:

- Fáciles de evaluar
- Únicos en cada una de sus realizaciones físicas
- Repetibilidad de su comportamiento (aunque con ruido)
- No se puede predecir su comportamiento hasta que no se implementa
- Si se copia o se altera, se detecta que no es genuino



Variabilidad genética:
Rasgos biométricos



Variabilidad del proceso de fabricación:
Circuitos que actúan como PUFs

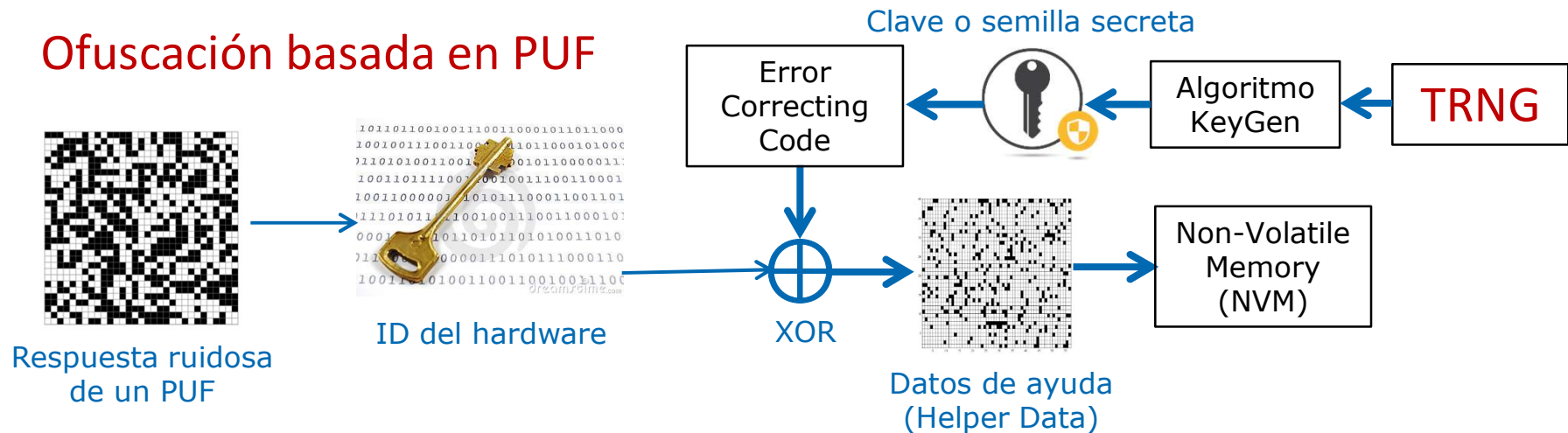
ID del hardware



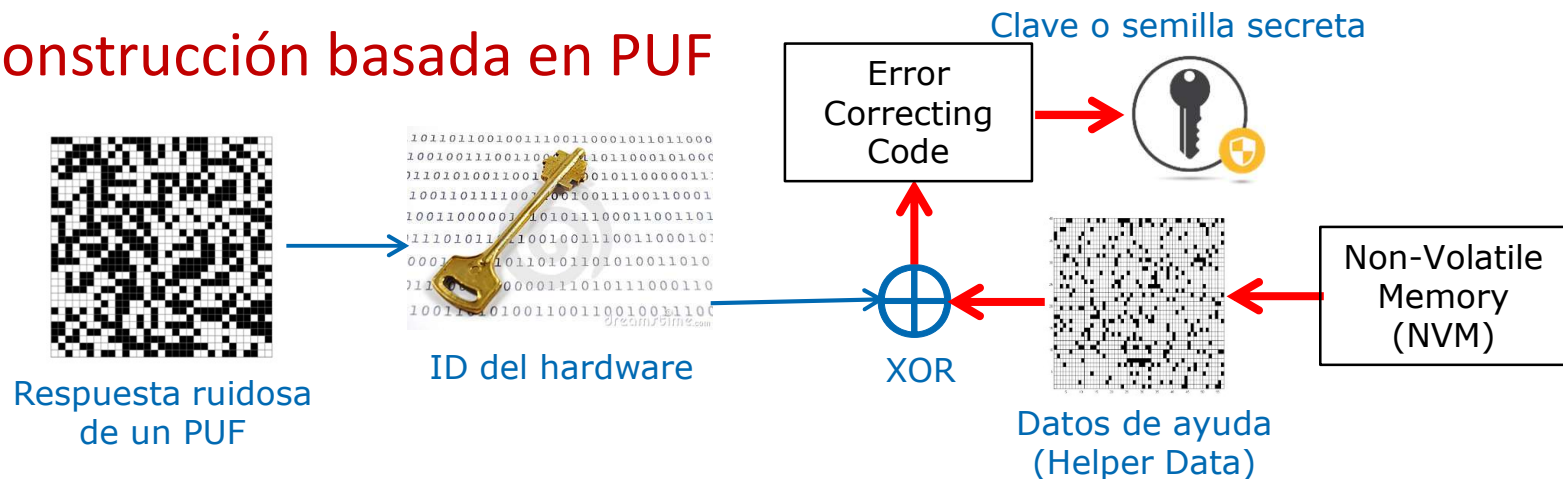
NECESIDAD DE SEGURIDAD HARDWARE

Generación y uso de claves criptográficas

Ofuscación basada en PUF



Reconstrucción basada en PUF



CONTENIDOS

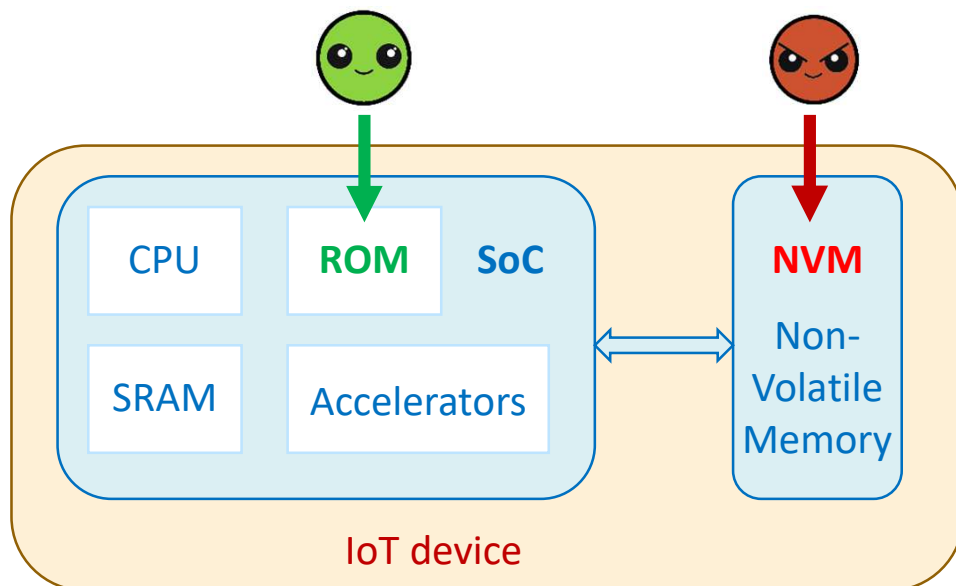
Necesidad de seguridad hardware

1. Soluciones con seguridad hardware
2. Generación y uso de claves criptográficas
- 3. Arranque seguro (secure boot)**
4. Actualización segura de firmware
5. Almacenamiento sellado de información
6. Atestación (remota) de plataformas
7. Conclusiones

NECESIDAD DE SEGURIDAD HARDWARE

Arranque seguro (secure boot)

- El proceso de arranque suele constar de varias etapas ejecutadas en cadena:
 - El primer código a ejecutar es una rutina (denominada en adelante BootROM) almacenada en una memoria programable de una sola vez (OTP) como la ROM interna del microcontrolador en el caso de un dispositivo IoT.
 - Los siguientes códigos a ejecutar suelen almacenarse en una memoria externa no volátil (NVM).



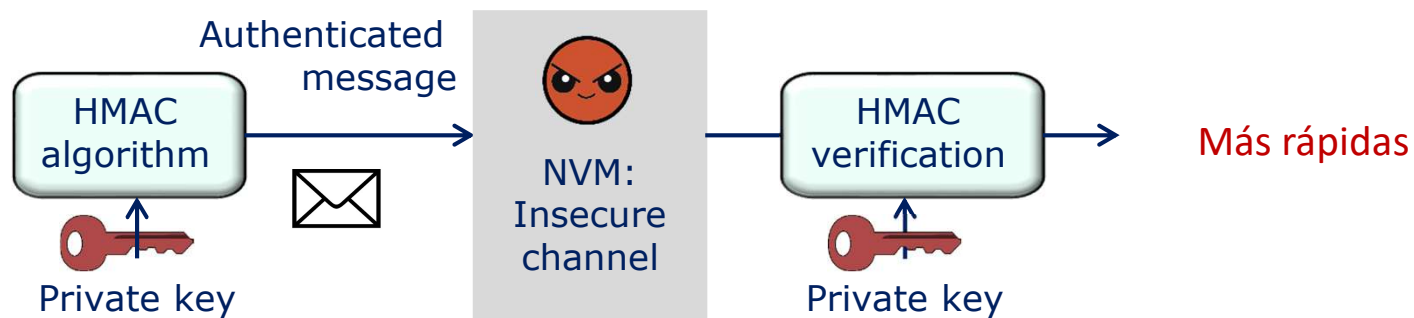
- La BootROM es intrínsecamente de confianza.
- ¡¡Los siguientes códigos deben ser verificados !!

NECESIDAD DE SEGURIDAD HARDWARE

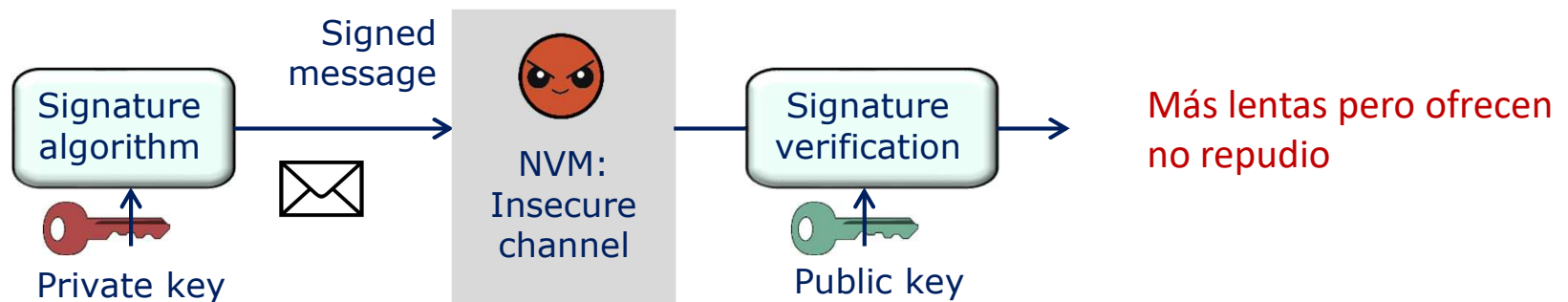
Arranque seguro (secure boot)

- Dos tipos de primitivas criptográficas para garantizar la integridad y autenticidad del código en el arranque seguro:

Simétricas, como un keyed-hash message authentication codes (**HMAC**)

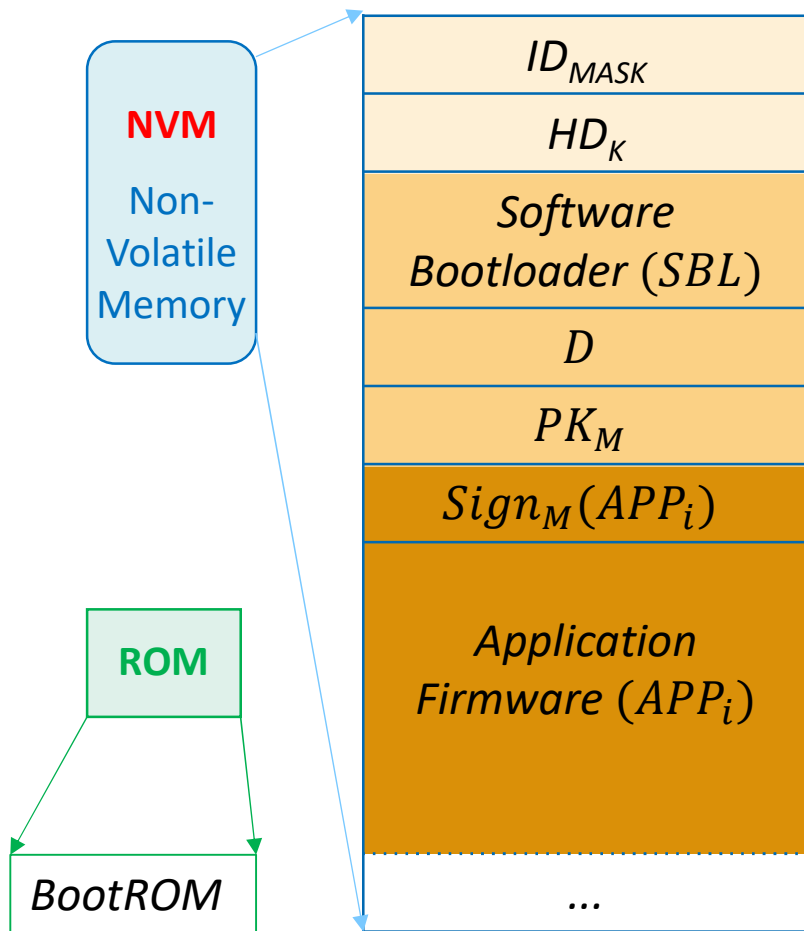


Asimétricas, típicamente usando **firmas digitales**



NECESIDAD DE SEGURIDAD HARDWARE

Arranque seguro (secure boot)



- Fase de registro

Se usa una SRAM PUF para ofuscar y recuperar al clave K , usada en el HMAC.

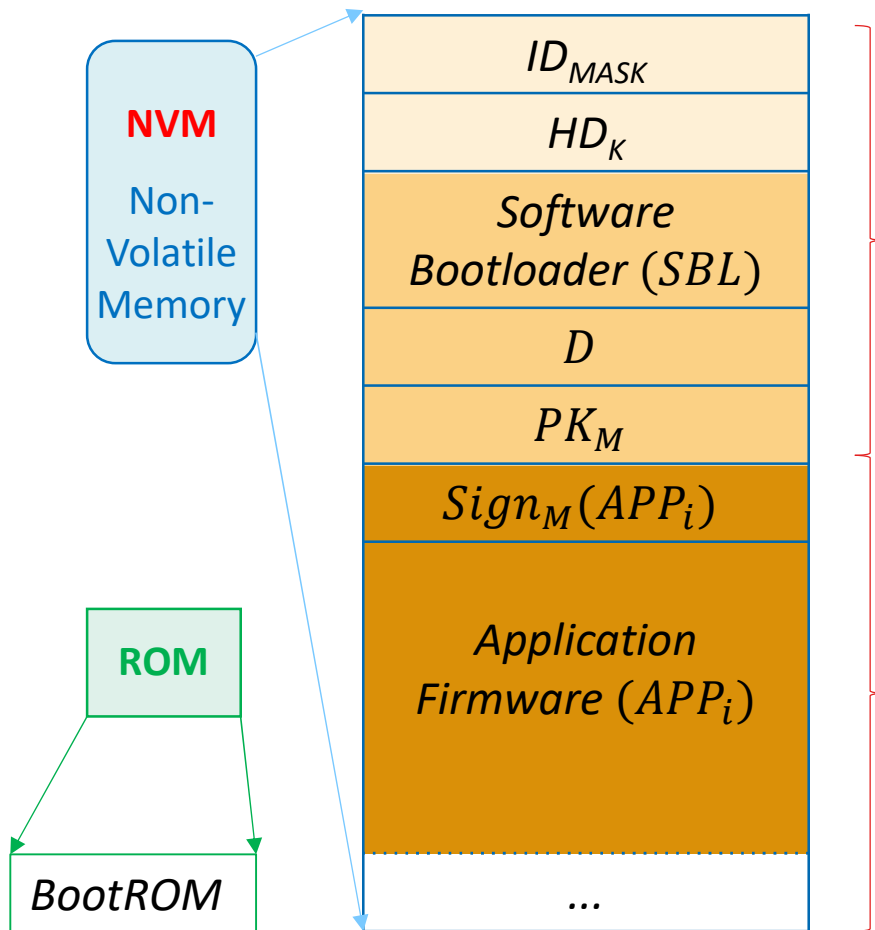
- La información que identifica a las celdas de la SRAM se almacena en ID_{MASK} .
- Los datos de ayuda para reconstruir la clave son HD_K .

El primer código a ejecutar, *BootROM*, almacenado en la ROM (es de confianza). El segundo código, *SBL*, se almacena en la NVM con su HMAC, D .

El *Application Firmware* lo firma el Fabricante. La firma ($Sign_M APP_i$) y la clave pública PK_M , permiten al dispositivo verificar la firma de APP_i en el arranque.

NECESIDAD DE SEGURIDAD HARDWARE

Arranque seguro (secure boot)



- Fase de arranque

Primero se ejecuta el *BootROM*:

- Se reconstruye la clave K , usando HD_K y la SRAM PUF identificada por ID_{MASK} .
- Se computa el $D' = HMAC_K(SBL || PK_M)$
- Si $D' = D$, se verifican el *SBL* y la PK_M .

Luego se ejecuta el *Software Bootloader*:

- Si se verifica la firma $Sign_M(APP_i)$, se verifica el *Application Firmware* y se ejecuta.

R. Román, I. Baturone, I. (2023). A Quantum-Resistant and Fast Secure Boot for IoT Devices Using Hash-Based Signatures and SRAM PUFs. In: Nayyar, A., Paul, A., Tanwar, S. (eds) The Fifth International Conference on Safety and Security with IoT . EAI/Springer Innovations in Communication and Computing. Springer, Cham.

CONTENIDOS

Necesidad de seguridad hardware

1. Soluciones con seguridad hardware
2. Generación y uso de claves criptográficas
3. Arranque seguro (secure boot)
- 4. Actualización segura de firmware**
5. Almacenamiento sellado de información
6. Atestación (remota) de plataformas
7. Conclusiones

NECESIDAD DE SEGURIDAD HARDWARE

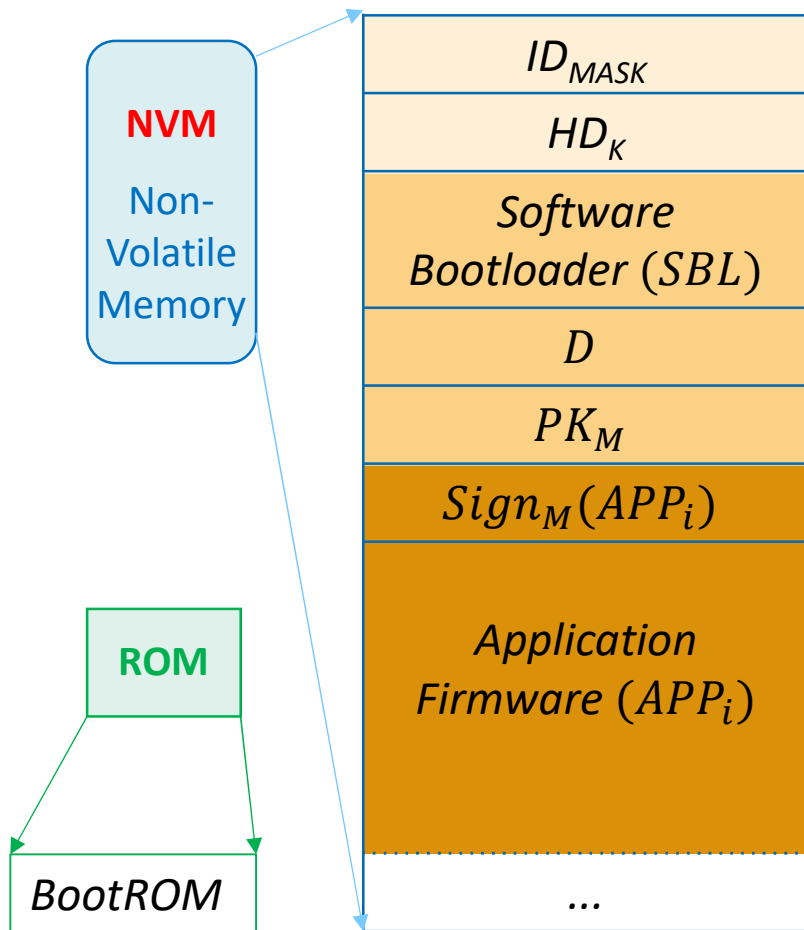
Actualización segura de firmware (Over The Air, OTA)



E. Ronen, A. Shamir, A. -O. Weingarten and C. O'Flynn, "IoT Goes Nuclear: Creating a ZigBee Chain Reaction," 2017 IEEE Symposium on Security and Privacy (SP), San Jose, CA, USA, 2017, pp. 195-212

NECESIDAD DE SEGURIDAD HARDWARE

Actualización segura de firmware (Over The Air, OTA)



Si el dispositivo recibe un requerimiento de *OTA firmware update*, con APP_j , firmado como $Sign_M(APP_j)$:

- El dispositivo verifica la firma con PK_M .

Si la firma se verifica:

- El dispositivo actualiza su *Application Firmware* a APP_j .
- Se calcula un nuevo HMAC como $D' = HMAC_K(SBL || PK_M)$
- Se almacena la nueva información en la lugar de la antigua, de modo que un arranque posterior pueda verificarla.

M.A. Prada-Delgado, A. Vazquez-Reyes, I. Baturone. "Trustworthy Firmware Update for Internet-of-Thing Devices Using Physical Unclonable Functions" Proc. Global IoT Summit, (2017).

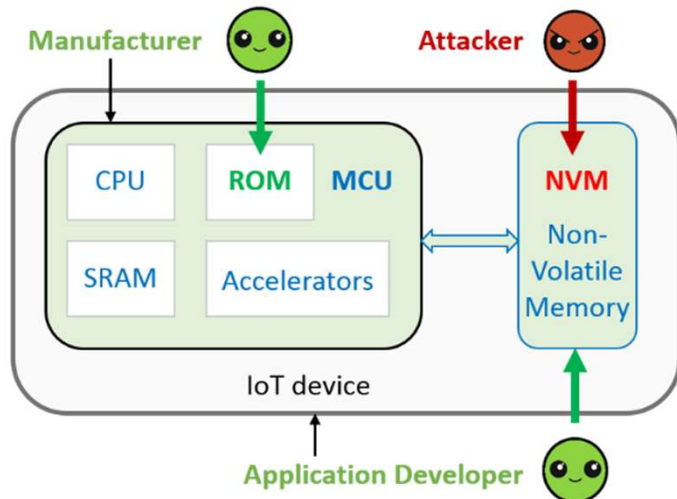
CONTENIDOS

Necesidad de seguridad hardware

1. Soluciones con seguridad hardware
2. Generación y uso de claves criptográficas
3. Arranque seguro (secure boot)
4. Actualización segura de firmware
- 5. Almacenamiento sellado de información**
6. Atestación (remota) de plataformas
7. Conclusiones

NECESIDAD DE SEGURIDAD HARDWARE

Almacenamiento sellado de información (caso IoT)



$seal()$
$unseal()$
$H(PK_{app})$

NVM
Non-Volatile
Memory

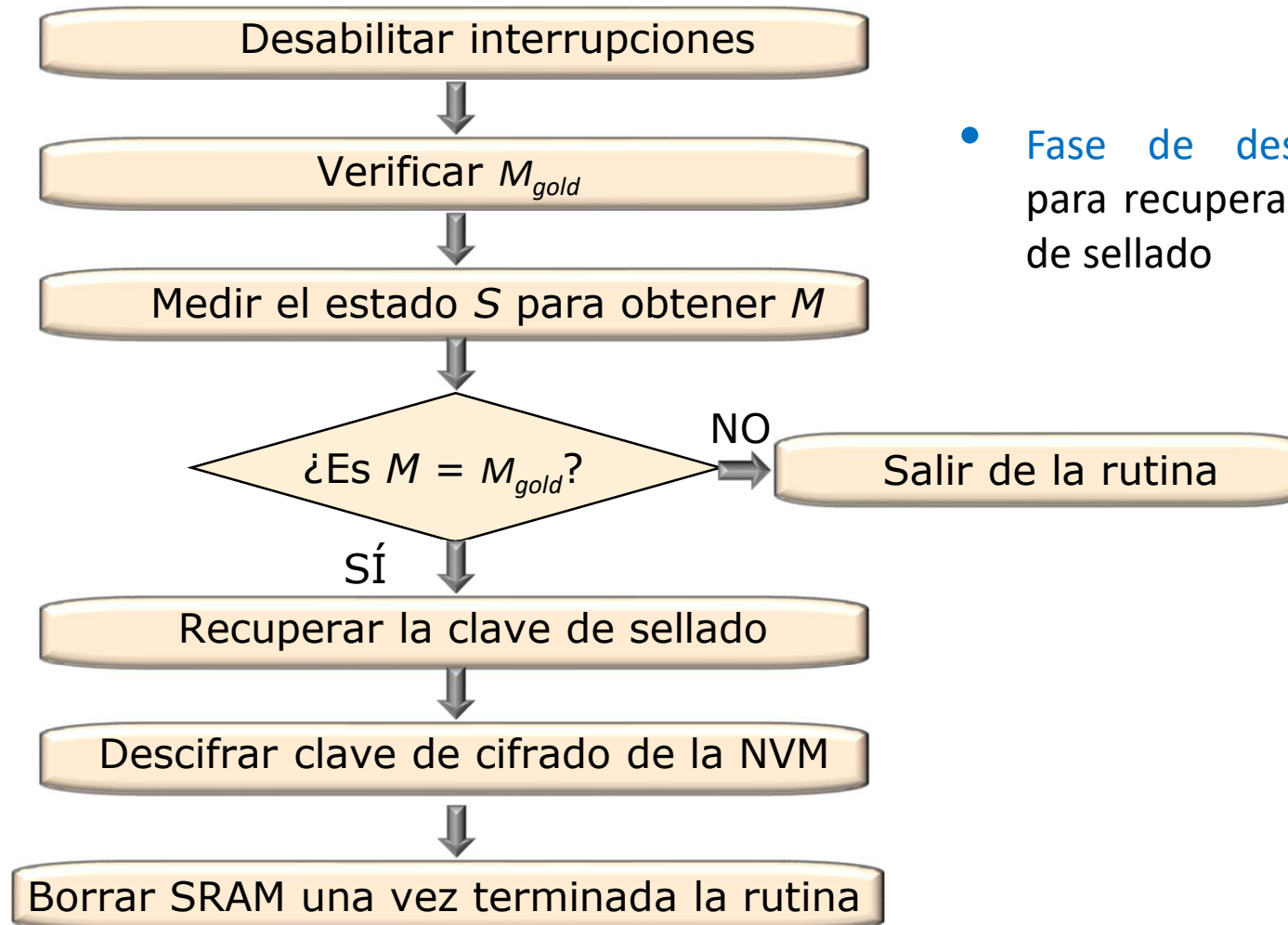
ID_{MASK}, HD_K
S, M_{gold}
PK_{app}
$Sign_{app}(ID_{MASK}, HD_K, S, M_{gold})$
$ENC(K), tag$
Application Firmware
...

Acuerdo entre el Fabricante
y el Desarrollador

- **Fase de fabricación:** el Fabricante escribe en la **ROM** las funciones $seal()$ y $unseal()$, el hash de la PK_{app} del Desarrollador, y caracteriza la SRAM PUF (con ID_{MASK} , y los datos de ayuda HD_K).
- **Fase de desarrollo:** el Desarrollador selecciona el estado S , calcula la medida M_{gold} , firma los datos, sella la clave de cifrado K , y escribe cifrada la **NVM**.

NECESIDAD DE SEGURIDAD HARDWARE

Almacenamiento sellado de información (caso IoT)



- Fase de despliegue: Pasos para recuperar y usar la clave de sellado

CONTENIDOS

Necesidad de seguridad hardware

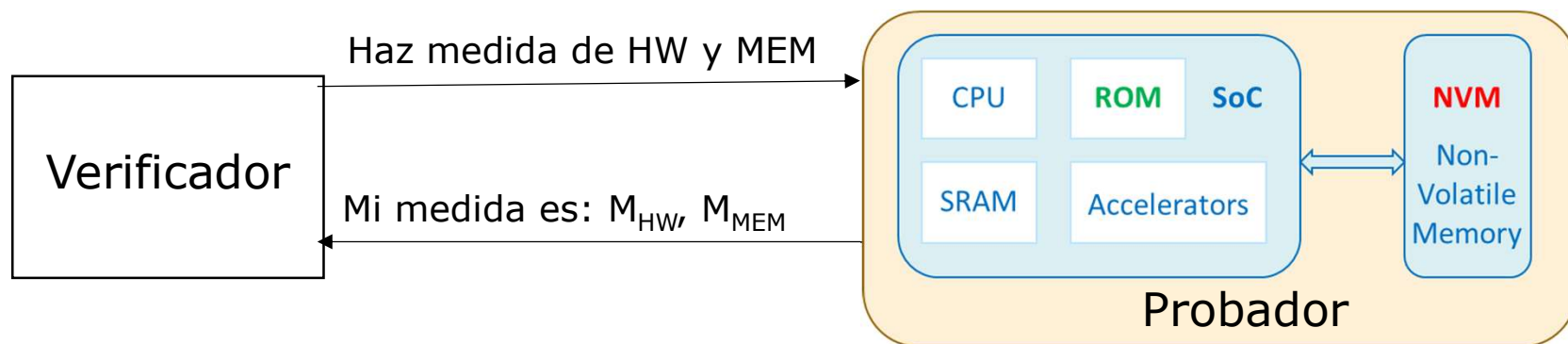
1. Soluciones con seguridad hardware
2. Generación y uso de claves criptográficas
3. Arranque seguro (secure boot)
4. Actualización segura de firmware
5. Almacenamiento sellado de información
- 6. Atestación (remota) de plataformas**
7. Conclusiones

NECESIDAD DE SEGURIDAD HARDWARE

Atestación (remota) de plataformas

- ¿Cómo sabe (un **Verificador**) si el código (software) de un dispositivo vulnerable (un **Probador**) ha sido modificado? ¿Y si el hardware es el original?
- Se necesita un **protocolo** de atestación entre Verificador y Probador:

El Verificador pide al Probador que haga una **medida** del código en su memoria (se aplica una función **hash** a su contenido) y se la envíe firmada (necesidad de una clave criptográfica en el Probador).

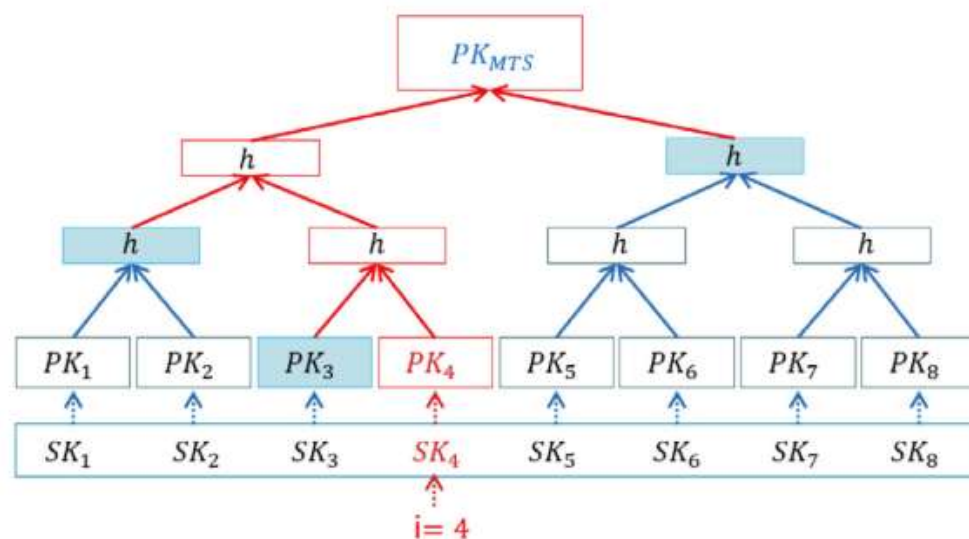


NECESIDAD DE SEGURIDAD HARDWARE

Atestación (remota) de plataformas

Se está evaluando la migración de la criptografía clásica a la post-cuántica.

Entre las firmas post-cuánticas, la primera recomendada por el NIST fue la XMSS (eXtended Merkle Signature Scheme)



NIST Special Publication 800-208

Recommendation for Stateful Hash-Based Signature Schemes

David A. Cooper
Daniel C. Apon
Quynh H. Dang
Michael S. Davidson
Morris J. Dworkin
Carl A. Miller

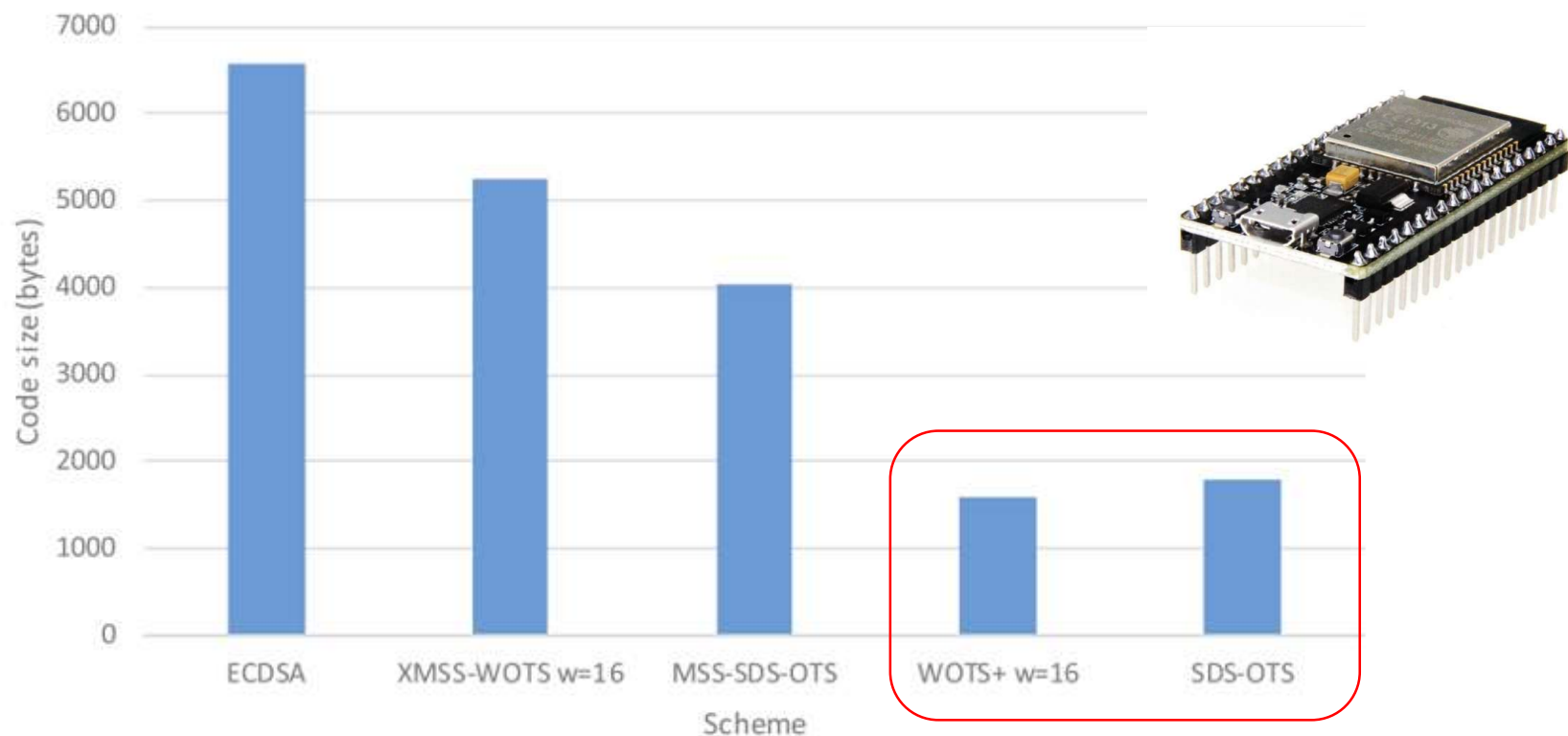
R. Román, R. Arjona, I. Baturone, "A lightweight remote attestation using PUFs and hash-based signatures for low-end IoT devices, Future Generation Computer Systems, Volume 148, Pages 425-435 (2023)

NECESIDAD DE SEGURIDAD HARDWARE

Atestación (remota) de plataformas

Resultados experimentales en una Pycom WiPy 3.0 Development Board, que incluye un microcontrolador ESP32, y se emplea en muchos dispositivos IoT.

No solo son importantes los tiempos de ejecución sino el tamaño de los códigos para poder almacenarlos en una ROM.



NECESIDAD DE SEGURIDAD HARDWARE

Conclusiones:

- El nivel más alto de confianza lo da la seguridad hardware.
- Entre las soluciones con seguridad hardware, destacan los módulos de seguridad hardware (HSMs), los elementos seguros (SE) y los entornos de ejecución segura (TEEs). La seguridad hardware en los dispositivos IoT está adaptada a la aplicación.
- Las claves criptográficas se necesitan para almacenar datos sensibles (claves de sellado) y transmitirlos y para autenticar plataformas a terceros (claves de atestación).
- Las PUFs (Physical Unclonable Functions) son muy interesantes para ofuscar y recuperar claves.
- El arranque seguro, la actualización segura de firmware, el almacenamiento sellado de la información y la atestación remota son funcionalidades que deben implementarse con seguridad hardware.